Open source licenses and compliance



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Introduction



Free software vs. open-source

- **Free software**: term defined by the *Free Software Foundation*, grants 4 freedoms
  - Freedom to use
  - Freedom to study
  - Freedom to copy
  - Freedom to modify and distribute modified copies
  - See https://www.gnu.org/philosophy/free-sw.html
- **Open Source**: term defined by the *Open Source Initiative*, with 10 criteria
  - See https://www.opensource.org/docs/osd
- *Free Software* movement insists more on ethics, while *Open Source* insists more on the technical advantages
- From a freedom standpoint, they are similar.



Open source licenses

- All free software/open-source licenses rely on *copyright law*
- Those licenses fall in two main categories
  - The copyleft licenses
  - The non-copyleft licenses, also called *permissive* licenses



Non-Copyleft VS Copyleft licenses

Non-Copyleft
(BSD, MIT, Apache, X11...)

You can
Use
Modify
Redistribute

You must
Provide license text
Attribution

Copyleft
(GPL, LGPL, AGPL...)

You can
Use
Modify
Redistribute

You must
Provide license text
Attribution
Make source code available



What is *copyleft*

- The concept of *copyleft* is to ask for reciprocity in the freedoms given to a user.
- You receive software under a copyleft license and redistribute it, modified or not → you must do so under the same license
  - Same freedoms to the new users
  - Incentive, but no obligation, to contribute back your changes instead of keeping them secret
- Copyleft is *not* the opposite of copyright!
- Non-copyleft licenses have no such requirements: modified versions can be made proprietary, but they still require attribution
- https://en.wikipedia.org/wiki/Copyleft



Non-copyleft licenses



Most common non-copyleft licenses

- **MIT**, **BSD 2 CLAUSE**
  - Very simple
  - Require to preserve the copyright notice
- **BSD 3 CLAUSE**
  - Adds a non-endorsement clause
- **Apache**
  - More complex
  - Includes a *patent grant*, a mechanism to prevent users of the licensed project from suing others based on patents related to the project



Copyleft licenses



GPL: GNU General Public License

- The flagship license of the GNU project
- Used by Linux, BusyBox, U-Boot, Barebox, GRUB, many projects from GNU
- Is a copyleft license
  - Requires derivative works to be released under the same license
  - Source code must be redistributed, including modifications
  - If GPL code is integrated in your code, your code must now be GPL-licensed
  - Only applies when redistribution takes place
- Also called **strong** copyleft license
  - Programs linked with a library released under the GPL must also be released under the GPL
  - Does not prevent GPL programs and non-GPL programs from co-existing or communicating in the same system
- https://www.gnu.org/licenses/gpl-2.0.en.html
- https://www.gnu.org/licenses/gpl-3.0.en.html
- https://en.wikipedia.org/wiki/GNU_General_Public_License



LGPL: GNU Lesser General Public License

- Used by *glibc*, *uClibc*, and many libraries
- Derived from the GPL license
- Also a copyleft license
- But a **weaker** copyleft license
  - Programs linked against a library under the LGPL do not need to be released under the LGPL and can be kept proprietary.
  - However, the user must keep the ability to update the library independently from the program.
  - Requires using dynamic linking, or in the case of static linking, to provide the object files to relink with the library
- https://www.gnu.org/licenses/lgpl-2.1.en.html
- https://www.gnu.org/licenses/lgpl-3.0.en.html
- https://en.wikipedia.org/wiki/GNU_Lesser_General_Public_License



GPL/LGPL: redistribution

- No obligation when the software is not distributed
  - You can keep your modifications secret until the product delivery
- It is then authorized to distribute binary versions, if one of the following conditions is met:
  - Convey the binary with a copy of the source on a physical medium
  - Convey the binary with a written offer valid for 3 years that indicates how to fetch the source code
  - Convey the binary with the network address of a location where the source code can be found
- In all cases, the attribution and the license must be preserved



GPL/LGPL: version 2 vs version 3

- GPLv2/LGPLv2 published in 1991, widely used in the open-source world for major projects
- GPLv3/LGPLv3 published in 2007, and adopted by some projects
- Main differences
  - More *legalese* and definitions to clarify the license
  - Explicit patent grant
  - Grace period of 30 days to get back into compliance instead of immediate termination
  - Anti-Tivoization clause
- Anti-Tivoization
  - Requirement that the user must be able to **run** the modified versions on the device
  - Need to provide *installation instructions*
  - Only required for *User products*, i.e. consumer devices
  - On-going debate on how strong this requirement is, and how difficult it is to comply with



GPL: v2, v3, v2 or later, v3 or later

- Some projects are released under *GPLv2 only*
  - Examples: Linux kernel, U-Boot
- Some projects are released under *GPLv3 only*
- Some projects are released under *GPLv2 or later*
  - The recipient can chose to apply either the terms of GPLv2, GPLv3 or any later version
- Some projects are released under *GPLv3 or later*
  - The recipient can chose to apply the terms of GPLv3 or any later version (none of which exists today)
  - Examples: GCC, Samba, Bash, GRUB
- Note: this logic applies similarly to the LGPL license.



Dual licensing

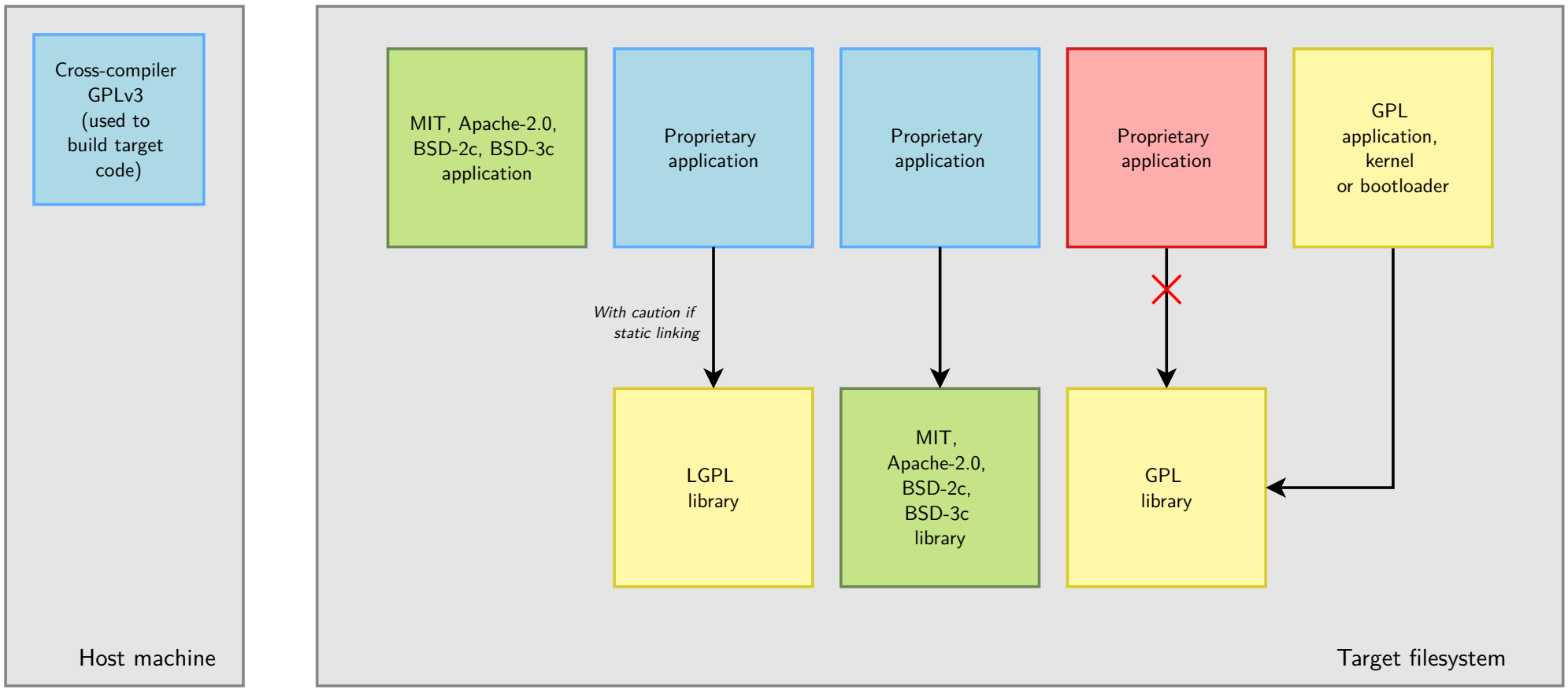
- Some companies use a *dual licensing* business model, mainly for software libraries
- Their software is offered under two licenses:
  - A strong copyleft license, typically GPL, to encourage adoption of the software by the open-source world, allow the development and distribution of GPL licensed applications based on this library
  - A commercial license, offered against a fee, which allows to develop and distribute proprietary applications based on this library.
- Examples: Qt (only parts), MySQL, wolfSSL, Asterisk, etc.



Is this free software?

- Most of the free software projects are covered by about 10 well-known licenses, so it is fairly easy for the majority of projects to get a good understanding of the license
- Check Free Software Foundation's opinion
  https://www.fsf.org/licensing/licenses/
- Check Open Source Initiative's opinion
  https://www.opensource.org/licenses
- Check the simplified license description on tl;drLegal
  https://www.tldrlegal.com
- Otherwise, read the license text

Licensing: examples





Best practices



Respect free software licenses

- Free Software is not public domain software, the distributors have obligations due to the licenses
- **Before** using a free software component, make sure the license matches your project constraints
- Make sure to keep your modifications and adaptations well-separated from the original version.
- Make sure to keep a complete list of the free software packages you use, and the version in use
- Buildroot and Yocto Project can generate this list for you!
  - Buildroot: `make legal-info`
  - Yocto: see the project documentation
- Conform to the license requirements before shipping the product to the customers.



Keeping changes separate

- When integrating existing open-source components in your project, it is sometimes needed to make modifications to them
  - Better integration, reduced footprint, bug fixes, new features, etc.
- Instead of mixing these changes, it is much better to keep them separate from the original component version
  - If the component needs to be upgraded, easier to know what modifications were made to the component
  - If support from the community is requested, important to know how different the component we're using is from the upstream version
  - Makes contributing the changes back to the community possible
- It is even better to keep the various changes made on a given component separate
  - Easier to review and to update to newer versions
- If possible, use the same version control system as the upstream project to maintain your changes.